

EXPERIMENT 7

Git for ML — Model Version Control & Dataset Versioning with DVC

Aim

To version large ML datasets and model artifacts alongside code using Git LFS and DVC (Data Version Control), keeping the Git repository lightweight while ensuring full reproducibility.

Theory

Plain Git is optimised for text and performs poorly with large binary files (datasets, model weights), because every version is stored in full, bloating the repository. Git LFS (Large File Storage) replaces large files with small text pointers and stores the actual content on a separate LFS server.

DVC (Data Version Control) is a Git-like tool built specifically for ML. It stores lightweight metadata (.dvc files) in Git while the actual data lives in remote storage (S3, GCS, Drive). This lets you 'git checkout' a commit and 'dvc checkout' the exact dataset/model version that corresponds to it, guaranteeing experiment reproducibility.

DVC also introduces pipelines (dvc.yaml) that define reproducible stages (prepare → train → evaluate) with tracked dependencies and outputs, so 'dvc repro' re-runs only the stages whose inputs changed.

Software / Tools Required

Git, Git LFS, DVC (pip install dvc), a sample dataset/model file.

Procedure (Step-by-Step)

Step 1. Initialise a repo and DVC together.

```
git init
```

Create a virtual env and activate

```
python3 -m venv .venv
source .venv/bin/activate
```

Install DVC and commit

```
pip install dvc
dvc init
git add .dvc .dvcignore
git commit -m "Initialize DVC"
```

Step 2. Create a sample dataset and start tracking it with DVC (not Git).

```
mkdir data
pip install numpy
python -c "import numpy as np; np.savetxt('data/train.csv', np.random.rand(1000,5), delimiter=',)"
dvc add data/train.csv
```

Step 3. Commit the lightweight .dvc pointer to Git; the real data stays out of Git.

```
git add data/train.csv.dvc data/.gitignore
git commit -m "Track dataset v1 with DVC"
git status
```

Step 4. Configure a DVC remote (e.g. a local folder standing in for cloud storage).

```
dvc remote add -d storage /tmp/dvc-store
dvc push
```

Step 5. Create a new version of the dataset and record it.

```
python -c "import numpy as np; np.savetxt('data/train.csv', np.random.rand(2000,5), delimiter=',,)"
dvc add data/train.csv
git commit -am "Dataset v2 (2000 rows)"
dvc push
git log --oneline
```

Step 6. Roll back to the previous dataset version by checking out the old commit.

```
git checkout HEAD~1 data/train.csv.dvc
dvc checkout # restores the v1 data file
```

Step 7. Track a trained model artifact the same way.

Create models Folder

```
mkdir -p models
```

we can create a dummy model artifact:

```
python -c "import pickle; pickle.dump({'model':'demo','accuracy':0.95}, open('models/model.pkl','wb'))"
```

then

```
dvc add models/model.pkl
git add models/model.pkl.dvc
git commit -m "Track model v1"
```

we need these files: create a folder src/train.py

```
mkdir -p src
```

we need these files: create a file in src named train .py and write this content

```
import pickle
import os
os.makedirs("models", exist_ok=True)
model = {"name": "Demo Model", "accuracy": 0.95}
with open("models/model.pkl", "wb") as f: pickle.dump(model, f)
print("Model trained and saved.")
```

Note if getting error remove the file from staged changes

```
git remove models/model.pkl.dvc
```

Step 8. Define a reproducible pipeline stage.

```
dvc stage add -n train -d src/train.py -d data/train.csv -o models/model.pkl python src/train.py
```

Step 9. Reproduce the pipeline; DVC skips stages whose inputs are unchanged.

```
dvc repro
```

Step 10. Compare metrics across experiments.

Note : use these commands in between to check for differentiation

```
dvc metrics show
dvc metrics diff
```

Expected Output

The Git repository stays small (only .dvc pointers are committed), while 'dvc push/pull' moves the actual data to remote storage. Checking out an old commit and running 'dvc checkout' restores the exact matching dataset version, confirming reproducible data/model versioning.

Conclusion

ML datasets and model artifacts were versioned with DVC, linking data versions to code commits so any experiment can be reproduced exactly — a core MLOps practice.

Viva Questions

1. Why is plain Git unsuitable for large datasets?
2. How does DVC keep the Git repository small?
3. What is stored in a .dvc file?
4. What is the difference between 'dvc push' and 'git push'?
5. What does a DVC pipeline stage track?